

UT1: Manejo de Ficheros (Texto, Binarios y XML)

Manual Exhaustivo de Persistencia Física, Flujos I/O, NIO.2 y Serialización en Java

MÓDULO FORMATIVO	UNIDAD DE TRABAJO	RESULTADOS DE APRENDIZAJE	PROFESOR & CURSO
Acceso a Datos (0486)	UT1 / UD01 (30 Horas)	RA1 (CE1.a, CE1.b, CE1.c, CE1.d, CE1.e)	Sergio Capdevila Díez · Curso 2026-2027

Estructura de Contenidos del Documento

1. 1. Introducción Contextualizada y Persistencia

2. 2. Conceptos Fundamentales de I/O y Streams

3. 3. Desarrollo Progresivo y Exhaustivo por Temas

4. 4. Ejemplos Prácticos Graduados (Básico a Avanzado)

5. 5. Código Funcional Paso a Paso: Gestor Empresarial

6. 6. Errores Habituales y Soluciones Técnicas

7. 7. Buenas Prácticas de la Industria por Niveles

8. 8. Matrices Comparativas de Tecnologías

9. 9. Resumen Clave de la Unidad

10. 10. Conceptos que Debes Dominar

11. 11. Glosario y 15 Preguntas de Repaso

1 Introducción Contextualizada y Persistencia

En cualquier sistema informático profesional, los datos almacenados en variables o estructuras en memoria RAM (objetos, arrays, colecciones) tienen un ciclo de vida efímero: desaparecen en cuanto la aplicación finaliza su ejecución, el proceso es detenido o el servidor físico se reinicia. Para conseguir que la información trascienda en el tiempo, es imprescindible implementar mecanismos de **persistencia** en medios no volátiles de almacenamiento secundario (discos de estado sólido SSD, discos duros mecánicos HDD o volúmenes de red NAS/SAN).

Utilidad Real en el Mundo Profesional

Aunque en cursos posteriores y proyectos de gran envergadura la persistencia principal recae sobre Sistemas Gestores de Bases de Datos Relacionales (SGBD) o motores NoSQL, el manejo de ficheros a bajo nivel continúa siendo la columna vertebral técnica de:

- **Registros de auditoría del sistema (Logs):** Generación y rotación de trazas de eventos en servidores web (Apache, Nginx) y microservicios con Logback o Log4j2.
- **Ficheros de configuración de entornos:** Parámetros de arranque del aplicativo en formatos de texto plano (`.properties`, `.env`, `.yaml`, `.json`).
- **Procesos ETL (Extract, Transform, Load) e Ingesta Masiva:** Descarga, análisis y consolidación masiva de millones de registros estructurados en ficheros delimitados (CSV, TSV).
- **Intercambio Seguro B2B y Facturación Electrónica:** Integración entre entidades públicas y bancarias mediante esquemas estándar XML (formato oficial *Facturae*, SEPA ISO 20022).

- ¿Qué conocimientos previos necesitas?** Para seguir con éxito esta unidad formativa debes dominar los fundamentos de Programación en Java de 1º de DAM:
- Programación orientada a objetos: definición de clases, constructores, atributos, visibilidad y métodos.
 - Uso de colecciones básicas de la Java Collections Framework: `List` , `ArrayList` y `Map` .
 - Gestión estructurada de excepciones mediante bloques `try-catch-finally` y propagación de errores con la cláusula `throws` .

2 Conceptos Fundamentales de I/O y Streams

2.1. La Metáfora Intuitiva del Stream (Flujo de Datos)

Para entender cómo se comunican las aplicaciones con el disco, piensa en una **tubería de agua** que conecta un embalse con un grifo. Tú no abres el grifo y recibes todo el volumen del embalse de golpe en una milésima de segundo; el agua viaja de manera continua, gota a gota o litro a litro, a través de la tubería.

En Java, un **Stream (Flujo)** es exactamente esa tubería: una secuencia ordenada, secuencial y unidireccional de datos que conecta un origen (fuente) con un destino (sumidero). La gran ventaja conceptual del flujo es que ****abstrae por completo el hardware****: para tu código Java, leer 100 bytes de un archivo local en un disco SSD, leerlos desde un puerto de red TCP o recibirlos del teclado utiliza exactamente las mismas llamadas a métodos.

2.2. Las Dos Grandes Jerarquías: Bytes frente a Caracteres

El paquete fundacional de Java (`java.io`) divide todos los flujos en dos árboles de clases completamente separados según el tipo elemental de dato que transportan:

CRITERIO DE CLASIFICACIÓN	FLUJOS DE BYTES (8 BITS / BINARIOS)	FLUJOS DE CARACTERES (16 BITS / UNICODE)
Unidad mínima de datos	byte (8 bits, valores de -128 a 127 / 0 a 255).	char (16 bits, código Unicode UTF-16).
Clase base de Entrada (Lectura)	<code>java.io.InputStream</code>	<code>java.io.Reader</code>
Clase base de Salida (Escritura)	<code>java.io.OutputStream</code>	<code>java.io.Writer</code>
Tipo de contenido idóneo	Imágenes (PNG, JPG), audio, ejecutables, PDFs, ficheros compilados <code>.class</code> .	Texto legible por humanos: archivos <code>.txt</code> , <code>.csv</code> , código fuente, <code>.xml</code> , <code>.json</code> .
Conversión de codificación	No aplica: los bytes se leen o escriben tal cual están físicamente.	Aplica traducción de bytes a caracteres usando un Charset (juego de caracteres).

2.3. Memoria Intermedia (Buffer) y el Patrón Decorador

Acceder a un dispositivo de almacenamiento físico es miles de veces más costoso en tiempo de CPU que operar en la memoria RAM. Si tu programa lee un archivo carácter a carácter invocando un método de disco en cada iteración, la JVM debe solicitar continuas interrupciones al sistema operativo (cambios de contexto de modo usuario a modo kernel), degradando el rendimiento de forma crítica.

💡 El Búfer en Memoria y el Patrón Decorador

Un **Búfer** es un bloque de memoria RAM de alta velocidad (típicamente de 8192 bytes / 8 KB) que actúa como almacén intermedio. Cuando pides el primer carácter, el sistema lee de golpe 8192 caracteres del disco y los guarda en RAM. Las siguientes 8191 lecturas se realizan instantáneamente desde la memoria sin tocar el disco. Java implementa esto mediante el **Patrón Decorador**: tomas un flujo básico de bajo nivel (por ejemplo `FileReader`) y lo envuelves dentro de una clase decoradora que añade el búfer transparente (`BufferedReader`).

2.4. Gestión de Recursos del Sistema Operativo: try-with-resources

Cada vez que abres un fichero, el núcleo del sistema operativo reserva un **descriptor de fichero (file handle)** en su tabla interna de procesos. Este recurso es limitado por el kernel (a menudo 1024 ficheros abiertos simultáneos en Linux). Si un programa en bucle abre flujos y no los cierra debidamente, se produce una ****fuga de recursos (resource leak)****, bloqueando el archivo en Windows e impidiendo que el servidor abra nuevas conexiones o ficheros con el error fatal:

```
java.io.IOException: Too many open files
```

Para erradicar este problema, desde Java 7 se utiliza la sentencia `try-with-resources`. Cualquier objeto instanciado dentro del paréntesis del `try (...)` que implemente la interfaz `java.lang.AutoCloseable` garantizará la ejecución de su método `close()` de forma automática y ordenada al salir del bloque, incluso si ocurre una excepción grave no controlada.

3 Desarrollo Progresivo y Exhaustivo por Temas

3.1. Ficheros de Texto Plano con la API Clásica (java.io)

Analicemos individualmente las clases fundamentales de la biblioteca clásica `java.io`, sus métodos más usados, ejemplos de código funcional y sus casos de aplicación práctica.

A. Clase `java.io.File`

Concepto: Representación abstracta del nombre y la ruta de un archivo o directorio en el sistema de archivos local. No permite leer ni escribir el contenido del archivo; sirve exclusivamente para consultar metadatos y realizar operaciones sobre el sistema de ficheros.

Instrucciones y Métodos Más Usados:

- `boolean exists()` : Retorna `true` si el archivo o carpeta existe físicamente en el disco.
- `boolean isFile()` y `boolean isDirectory()` : Verifican si el elemento es un archivo regular o una carpeta.
- `String getName()` : Retorna el nombre simple del fichero con su extensión (ejemplo: `"datos.txt"`).
- `String getAbsolutePath()` : Retorna la ruta completa normalizada desde la raíz del sistema.
- `long length()` : Devuelve el tamaño físico del archivo en bytes (para directorios retorna un valor indefinido).
- `boolean createNewFile() throws IOException` : Crea un nuevo archivo vacío de forma atómica; retorna `true` si se creó con éxito.
- `boolean mkdir()` y `boolean mkdirs()` : Crea la carpeta. `mkdirs()` crea además todos los directorios padres intermedios que no existan.
- `boolean delete()` : Borra el archivo o carpeta. Para borrar una carpeta, esta debe estar completamente vacía.
- `File[] listFiles()` : Retorna un array con todos los archivos y subcarpetas que contiene el directorio.

GestionMetadatosFile.java

JAVA 21

```
import java.io.File;
import java.io.IOException;

public class GestionMetadatosFile {
    public static void inspeccionarRuta(String ruta) {
        File f = new File(ruta);

        System.out.println("Ruta analizada: " + f.getAbsolutePath());
        if (f.exists()) {
            System.out.println("¿Es directorio?: " + f.isDirectory());
            System.out.println("¿Es archivo?: " + f.isFile());
            System.out.println("Tamaño físico: " + f.length() + " bytes");
            System.out.println("Permisos - Lectura: " + f.canRead() + " | Escritura: " + f.canWrite());
        } else {
            System.out.println("El elemento no existe en disco.");
        }
    }
}
```

B. Clases `FileReader` y `FileWriter`

Concepto: Flujos de conexión directa con disco físico para leer y escribir caracteres.

Instrucciones y Métodos Más Usados:

- `new FileReader(File archivo, Charset charset)` : Abre el fichero para lectura especificando el juego de caracteres obligatorio.

- `int read()` : Lee un único carácter y lo devuelve como un `int` (su código Unicode de 0 a 65535). Retorna `-1` al alcanzar el final del archivo (EOF).
- `int read(char[] cbuf)` : Lee un bloque de caracteres y los almacena en el array `cbuf`, retornando la cantidad leída.
- `new FileWriter(File archivo, Charset charset, boolean append)` : Abre el archivo para escritura. Si `append` es `true`, añade al final sin borrar el contenido existente.
- `void write(String str)` : Escribe una cadena de texto completa.
- `void flush()` : Fuerza el vaciado inmediato de los datos en disco físico.

C. Clases Decoradoras: `BufferedReader` y `BufferedWriter`

Concepto: Envuelven a un `Reader` o `Writer` añadiendo una memoria intermedia de 8 KB. Proporcionan métodos de alto nivel orientados al tratamiento por líneas de texto.

Instrucciones y Métodos Más Usados:

- `String readLine() throws IOException` : Lee una línea completa de texto. Descarta automáticamente los delimitadores de salto de línea (`\r` o `\n`). Devuelve la cadena o `null` si se ha alcanzado el Fin de Fichero (EOF).
- `void newline() throws IOException` : Escribe el salto de línea adecuado según el sistema operativo donde corre el programa (CRLF en Windows, LF en Linux/macOS).

CopiaBufferSegura.java

JAVA 21

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class CopiaBufferSegura {
    public static void copiarConFiltro(File origen, File destino, String prefijoFiltro) {
        // Bloque try-with-resources: asegura el cierre implícito de ambos flujos
        try (
            BufferedReader br = new BufferedReader(new FileReader(origen, StandardCharsets.UTF_8));
            BufferedWriter bw = new BufferedWriter(new FileWriter(destino, StandardCharsets.UTF_8))
        ) {
            String linea;
            int numLinea = 1;

            // Patrón estándar de lectura línea a línea en Java
            while ((linea = br.readLine()) != null) {
                if (linea.startsWith(prefijoFiltro)) {
                    bw.write "[" + numLinea + " " + linea);
                    bw.newLine(); // Salto de línea portable
                }
                numLinea++;
            }
            // Al cerrarse el bloque se ejecuta automáticamente bw.flush() y bw.close()
        } catch (FileNotFoundException e) {
            System.err.println("Error: Archivo de origen no localizable: " + e.getMessage());
        } catch (IOException e) {
            System.err.println("Error crítico de Entrada/Salida: " + e.getMessage());
        }
    }
}
```

Trabajo Síncrono en Aula: Actividad 1.1 y 1.2

Abre tu IDE de desarrollo (Eclipse / IntelliJ) y realiza las **Actividades 1.1 y 1.2 del Cuaderno de Actividades de Aula** para practicar la inspección de archivos con `File` y la lectura por bloques con `BufferedReader`.

3.2. La Revolución de la API Moderna Java NIO.2 (java.nio.file)

Introducida en Java 7 y consolidada en las versiones modernas de Java SE, la biblioteca NIO.2 reemplaza los defectos estructurales de `java.io.File` aportando soporte multiplataforma completo, manejo atómico de archivos y programación funcional mediante Streams.

A. Interfaz `java.nio.file.Path` y Métodos Factoría

Concepto: Objeto inmutable que representa la localización de un fichero en el árbol de directorios de manera agnóstica al sistema operativo.

Instrucciones y Métodos Más Usados:

- `Path.of(String primero, String... mas)` : (Recomendado en Java 11+) Construye un `Path` ensamblando las partes sin importar si el SO usa barras inclinadas `/` o invertidas `\`.
- `Path getFileName()` : Retorna el nombre final del recurso.
- `Path getParent()` : Retorna la ruta del directorio contenedor.
- `Path toAbsolutePath()` : Resuelve la ruta relativa respecto al directorio de trabajo actual de la aplicación.
- `Path normalize()` : Elimina redundancias como puntos individuales `.` (directorio actual) y dobles puntos `..` (directorio padre).
- `Path resolve(String otraRuta)` : Concatena dos rutas de forma segura.
- `Path relativize(Path origen)` : Calcula la ruta relativa necesaria para desplazarse desde un `Path` hasta otro.

B. Clase de Utilidad `java.nio.file.Files`

Concepto: Clase estática compuesta enteramente por métodos de utilidad que operan directamente sobre objetos `Path`. A diferencia de `java.io.File` (que devolvía `false` cuando algo fallaba en silencio), `Files` lanza excepciones precisas y descriptivas (`NoSuchFileException`, `DirectoryNotEmptyException`, `FileAlreadyExistsException`).

Operaciones del Sistema de Archivos:

- `boolean exists(Path p)` : Verifica existencia.
- `Path createDirectories(Path p)` : Crea toda la jerarquía de directorios inexistentes.
- `void delete(Path p) throws IOException` : Borra el fichero o lanza excepción si no existe.
- `Path copy(Path origen, Path destino, CopyOption... opciones)` : Copia el fichero. Permite la opción `StandardCopyOption.REPLACE_EXISTING` para sobrescribir.
- `Path move(Path origen, Path destino, CopyOption... opciones)` : Mueve o renombra. Con `StandardCopyOption.ATOMIC_MOVE` asegura que la operación no deje archivos a medio escribir ante caídas de corriente.
- `long size(Path p)` : Retorna el tamaño exacto en bytes.

C. Métodos Atómicos de Lectura y Escritura en Files

Para archivos de tamaño pequeño o moderado (menos de 20-50 MB), `Files` resuelve la lectura o volcado en una única línea de código, abriendo y cerrando los descriptores de forma automática:

- `String readString(Path ruta, Charset cs)` : Lee todo el archivo y lo vuelca directamente en un objeto `String`.
- `Path writeString(Path ruta, CharSequence texto, Charset cs, OpenOption... opciones)` : Escribe el texto en el archivo. Admite opciones de apertura como `StandardOpenOption.CREATE` y `StandardOpenOption.APPEND`.
- `List<String> readAllLines(Path ruta, Charset cs)` : Carga todas las líneas del fichero en una colección `List<String>` en memoria.

D. Procesamiento Funcional de Archivos Gigantes con `Files.lines()`

¿Qué ocurre si el archivo pesa 5 Gigabytes? Usar `readAllLines()` provocaría instantáneamente un error catastrófico `OutOfMemoryError` al intentar almacenar 5 GB de objetos en el Heap de la JVM. Para ficheros masivos se utiliza `Files.lines()`, que genera un flujo perezoso (*Lazy Stream*): las líneas se leen una a una del disco únicamente cuando el pipeline de operaciones funcionales las solicita.

⚠ Regla Crítica: El Stream de Files.lines() DEBE cerrarse con try-with-resources

A diferencia de los Streams habituales de colecciones (como `lista.stream()`), el `Stream<String>` que produce `Files.lines()` mantiene abierto un descriptor de fichero en el núcleo del sistema operativo. Por tanto, **es obligatorio envolverlo dentro de un `try-with-resources`** para garantizar que el archivo se libere al finalizar la lectura.

AnalisisLogStreamingNIO.java

JAVA 21

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.stream.Stream;

public class AnalisisLogStreamingNIO {
    public static void auditarLogServidor(Path rutaLog) {
        // Files.lines abre el canal del SO: DEBE ir en el try()
        try (Stream<String> lineas = Files.lines(rutaLog, StandardCharsets.UTF_8)) {

            long totalErroresCriticos = lineas
                .filter(linea -> linea.contains("[ERROR]") || linea.contains("[FATAL]"))
                .filter(linea -> linea.contains("DatabaseConnectionException"))
                .peek(System.out::println) // Muestra cada línea coincidente
                .count(); // Operación terminal que consume el flujo de forma perezosa

            System.out.println("Total de fallos de base de datos encontrados: " + totalErroresCriticos);

        } catch (IOException e) {
            System.err.println("Fallo al procesar el archivo con NIO.2: " + e.getMessage());
        }
    }
}
```

3.3. Juegos de Caracteres (Charsets) y Trampas de Codificación

Un disco físico no entiende de letras, vocales con tilde ni caracteres especiales; únicamente almacena combinaciones de ceros y unos (bytes). La traducción entre una secuencia binaria y un carácter alfanumérico legible se rige por una tabla de correspondencia matemática denominada **Juego de Caracteres (Charset)**.

- **ASCII (7 bits)**: Solo cubre 128 caracteres estándar en inglés (sin tildes, ñ ni símbolos como €).
- **ISO-8859-1 / Windows-1252 (ANSI, 8 bits)**: Añade caracteres de idiomas de Europa Occidental, pero asigna códigos incompatibles con otros alfabetos.
- **UTF-8 (Longitud variable, 1 a 4 bytes)**: El estándar absoluto de la industria moderna. Representa cualquier carácter del estándar universal Unicode siendo compatible hacia atrás con ASCII. Un carácter común en inglés ocupa 1 byte, pero una 'á' o una 'ñ' ocupan 2 bytes, y el símbolo del euro '€' ocupa 3 bytes.

❌ La Trampa del Charset por Defecto y el Mojibake

Si creas un `new FileReader(archivo)` o invocas `Files.readAllLines(archivo)` sin indicar el juego de caracteres, Java utiliza el valor del sistema anfitrión: `Charset.defaultCharset()`.

- En un equipo de desarrollo Windows en español, el valor por defecto suele ser `windows-1252`.
- En un servidor Linux de despliegue o un contenedor Docker, suele ser `UTF-8`.

Consecuencia: Un archivo con caracteres españoles ("Camión", "Diseño") creado en Windows se leerá en Linux como texto corrupto (*mojibake*: `"CamíÃ³n"` o `"DiseÃ±o"`).

Solución Obligatoria: Declarar siempre la constante `StandardCharsets.UTF_8`.

3.4. Ficheros Binarios y Flujos de Bytes (java.io)

Cuando los datos no representan texto sino estructuras de información binaria (ficheros comprimidos, imágenes, gráficos o valores numéricos primitivos), se utilizan flujos derivados de `InputStream` y `OutputStream`. Almacenar datos en binario elimina el sobrecoste de parseo textual y ahorra espacio físico en disco (un entero `int` siempre ocupa 4 bytes exactos frente a los 10 bytes que requeriría en texto).

A. Clases Base: `FileInputStream` y `FileOutputStream`

Concepto: Flujos fundamentales de bajo nivel que establecen una conexión directa con el canal físico del sistema operativo para leer y escribir secuencias de bytes individuales (valores numéricos de 0 a 255).

INSTRUCCIÓN / MÉTODO	PARÁMETROS	RETORNO	QUÉ HACE EXACTAMENTE EN DISCO / MEMORIA
<code>int read() throws IOException</code>	Ninguno	<code>int</code> (0 a 255 o -1)	Lee el siguiente byte físico del archivo y avanza el cabezal 1 posición. Devuelve -1 al alcanzar el Fin de Fichero (EOF).
<code>int read(byte[] b, int off, int len)</code>	<code>byte[] b</code> , <code>int off</code> , <code>int len</code>	<code>int</code> (bytes leídos)	Lee un bloque de hasta <code>len</code> bytes del archivo y los transfiere directamente a la memoria RAM en el array <code>b</code> comenzando en el índice <code>off</code> .
<code>void write(int b) throws IOException</code>	<code>int b</code>	<code>void</code>	Escribe los 8 bits menos significativos del entero en el archivo físico.
<code>void write(byte[] b, int off, int len)</code>	<code>byte[] b</code> , <code>int off</code> , <code>int len</code>	<code>void</code>	Vuelca en bloque <code>len</code> bytes del array <code>b</code> hacia el medio de almacenamiento físico.

B. Flujos de Datos Tipados: `DataOutputStream` y `DataInputStream`

Concepto: Decoradores de flujos binarios que implementan las interfaces `DataOutput` y `DataInput`. Permiten escribir y leer variables primitivas de Java (`int`, `double`, `boolean`, etc.) y cadenas en formato UTF-8 modificado, utilizando el orden de bytes estándar de red (**Big-Endian**: el byte más significativo se escribe primero).

INSTRUCCIÓN DE ESCRITURA	INSTRUCCIÓN DE LECTURA	BYTES FÍSICOS	QUÉ HACE LA INSTRUCCIÓN EXACTAMENTE
<code>dos.writeByte(int v)</code>	<code>dis.readByte()</code>	1 byte	Escribe o lee 8 bits con signo (-128 a 127). En lectura lanza <code>EOFException</code> al final.
<code>dos.writeShort(int v)</code>	<code>dis.readShort()</code>	2 bytes	Escribe o lee 16 bits de un entero corto con signo.
<code>dos.writeInt(int v)</code>	<code>dis.readInt()</code>	4 bytes	Escribe o lee 32 bits de un entero Java estándar. Ocupa siempre 4 bytes fijos.
<code>dos.writeLong(long v)</code>	<code>dis.readLong()</code>	8 bytes	Escribe o lee 64 bits de un entero largo.
<code>dos.writeDouble(double v)</code>	<code>dis.readDouble()</code>	8 bytes	Escribe o lee 64 bits bajo el estándar IEEE 754 de coma flotante de doble precisión.
<code>dos.writeBoolean(boolean v)</code>	<code>dis.readBoolean()</code>	1 byte	Escribe <code>0x01</code> para true o <code>0x00</code> para false.
<code>dos.writeChar(int v)</code>	<code>dis.readChar()</code>	2 bytes	Escribe o lee los 16 bits de un carácter Unicode.
<code>dos.writeUTF(String str)</code>	<code>dis.readUTF()</code>	$\$2 + N\$$ bytes	Escribe primero 2 bytes indicando la longitud en bytes del texto, seguidos de los caracteres en UTF-8.
<code>dos.flush()</code>	-	-	Fuerza el vaciado inmediato de cualquier dato pendiente en el búfer de memoria hacia el disco.

✗ Regla de Oro: El Contrato de Simetría y la Captura de `EOFException`

1. Simetría obligatoria: Los datos binarios no tienen separadores. Debes leer exactamente en el mismo orden e invocar las mismas instrucciones con las que escribiste. Si escribes `writeInt()` seguido de `writeDouble()`, pero al leer invocas primero `readDouble()`, Java mezclará los 4 bytes del entero con 4 bytes del double, corrompiendo la posición del puntero.

2. Detección de Fin de Archivo: `readInt()` o `readDouble()` no pueden devolver `-1` ni `null` porque esos son números válidos. Por tanto, **avisan del final lanzando una excepción** `java.io.EOFException`. El bucle de lectura debe ser `while(true)` capturando canónicamente dicha excepción.

```
import java.io.*;

public class GestorTelemetryDataStream {

    public static void guardarMediciones(File archivo) {
        // Envolvemos FileOutputStream con BufferedOutputStream y DataOutputStream
        try (DataOutputStream dos = new DataOutputStream(
            new BufferedOutputStream(new FileOutputStream(archivo)))) {

            // Registro 1: ID sensor (int), Nombre (UTF), Temperatura (double), Alarma (boolean)
            dos.writeInt(101);           // 4 bytes
            dos.writeUTF("Caldera Central"); // 2 bytes cabecera + 15 bytes texto = 17 bytes
            dos.writeDouble(82.45);      // 8 bytes
            dos.writeBoolean(true);      // 1 byte (alarma activa)

            // Registro 2: Sensor de refrigeración
            dos.writeInt(102);           // 4 bytes
            dos.writeUTF("Rack Servidores A"); // 2 bytes + 17 bytes = 19 bytes
            dos.writeDouble(19.20);      // 8 bytes
            dos.writeBoolean(false);     // 1 byte (sin alarma)

            dos.flush();
            System.out.println("✅ Datos binarios volcados con éxito a disco.");

        } catch (IOException e) {
            System.err.println("Error al escribir datos tipados: " + e.getMessage());
        }
    }

    public static void leerMediciones(File archivo) {
        // Leemos con DataInputStream controlando el final mediante EOFException
        try (DataInputStream dis = new DataInputStream(
            new BufferedInputStream(new FileInputStream(archivo)))) {

            System.out.println("📖 Registros binarios recuperados:");
            while (true) {
                // Leemos en el orden estricto de escritura
                int id = dis.readInt();
                String nombre = dis.readUTF();
                double temp = dis.readDouble();
                boolean alarma = dis.readBoolean();

                System.out.printf(" Sensor [%d] %-18s | Temp: %6.2f °C | Alarma: %s\n",
                    id, nombre, temp, (alarma ? "⚠️ Sí" : "OK"));
            }

        } catch (EOFException e) {
            // Fin natural del archivo binario
            System.out.println("🏁 Fin de fichero alcanzado correctamente.");
        } catch (IOException e) {
            System.err.println("Error de lectura: " + e.getMessage());
        }
    }
}
```

Trabajo Síncrono en Aula: Actividad 2.1

Abre tu IDE y resuelve la **Actividad 2.1 del Cuaderno de Actividades de Aula** implementando un lector/escritor de mediciones de sensores con captura de `EOFException`.

3.5. Ficheros de Acceso Aleatorio (RandomAccessFile)

En los flujos secuenciales convencionales, para acceder al registro número 10.000 es obligatorio leer primero los 9.999 anteriores. La clase `java.io.RandomAccessFile` rompe esta restricción: modela el fichero como un ****gran array de bytes persistente en disco****, permitiendo desplazar un cabezal de lectura/escritura (**Puntero de Archivo o File Pointer**) de forma instantánea a cualquier coordenada en bytes.

A. Modos de Apertura en el Constructor

- `new RandomAccessFile(file, "r")` : Modo solo lectura (*Read-Only*). Si el fichero no existe, lanza `FileNotFoundException` . Cualquier intento de escritura lanza `IOException` .
- `new RandomAccessFile(file, "rw")` : Modo lectura y escritura (*Read-Write*). Si el fichero no existe en disco, lo crea automáticamente. Si existe, no borra su contenido: sitúa el puntero en el byte 0.
- `new RandomAccessFile(file, "rws")` y `"rwd"` : Modos sincronizados donde cada operación de escritura se sincroniza atómicamente a nivel de hardware en el disco físico.

B. Ficha Técnica de Instrucciones de Posicionamiento y Operación

INSTRUCCIÓN / MÉTODO	PARÁMETROS	RETORNO	QUÉ HACE EXACTAMENTE EN EL ARCHIVO FÍSICO
<code>raf.seek(long pos)</code>	<code>long pos</code> (offset en bytes)	<code>void</code>	Mueve el puntero físico exactamente al byte número <code>pos</code> . La posición inicial es 0. Permite saltar más allá del final del archivo para agrandarlo.
<code>raf.getFilePointer()</code>	Ninguno	<code>long</code>	Consulta y devuelve el índice exacto del byte donde se encuentra el cabezal de lectura/escritura en ese instante.
<code>raf.length()</code>	Ninguno	<code>long</code>	Devuelve el tamaño total en bytes del fichero físico.
<code>raf.setLength(long newLength)</code>	<code>long newLength</code>	<code>void</code>	Trunca o agranda el archivo forzando el tamaño especificado. Si se reduce, los datos sobrantes se eliminan irreversiblemente.
<code>raf.skipBytes(int n)</code>	<code>int n</code>	<code>int</code> (bytes saltados)	Avanza el puntero <code>n</code> bytes hacia adelante sin leer su contenido en memoria RAM.
<code>raf.writeInt(int v)</code>	<code>int v</code>	<code>void</code>	Escribe 4 bytes y avanza automáticamente el puntero 4 bytes hacia adelante.
<code>raf.readInt()</code>	Ninguno	<code>int</code>	Lee 4 bytes desde la posición actual y avanza el puntero 4 bytes.
<code>raf.writeDouble(double v)</code>	<code>double v</code>	<code>void</code>	Escribe 8 bytes y avanza el puntero 8 bytes.
<code>raf.readDouble()</code>	Ninguno	<code>double</code>	Lee 8 bytes desde la posición actual y avanza el puntero 8 bytes.
<code>raf.writeChars(String s)</code>	<code>String s</code>	<code>void</code>	Escribe cada carácter de la cadena como 2 bytes Unicode (formato fijo, sin bytes de longitud).
<code>raf.readChar()</code>	Ninguno	<code>char</code>	Lee 2 bytes y reconstruye el carácter Unicode avanzando el puntero 2 bytes.

C. La Necesidad de Registros de Longitud Fija y Cálculo de Offsets

Para que la función `seek()` pueda aterrizar con precisión matemática en el registro número N sin necesidad de leer los anteriores, **cada registro del archivo debe medir exactamente la misma cantidad de bytes**:

$$\text{Offset en bytes} = \text{Índice del Registro} \times \text{Tamaño Fijo del Registro}$$

⚠ Por qué `writeUTF()` está prohibido en registros fijos de `RandomAccessFile`

Si escribes un String con `writeUTF()`, los bytes que ocupa variarán según el contenido del texto: `"Ana"` ocupará 5 bytes (2 de cabecera + 3 caracteres), mientras que `"Constantinopla"` ocupará 16 bytes. Esta variabilidad desajusta la cuadrícula y destruye la alineación de todos los registros siguientes.

Solución Técnica Obligatoria: Fijar siempre una cantidad fija de caracteres mediante `StringBuffer.setLength()` y escribirla con `writeChars()`.

```

import java.io.File;
import java.io.IOException;
import java.io.RandomAccessFile;

public class AlmacenArticulosRAF {

    // Estructura fija: ID (4 bytes) + Nombre (10 chars = 20 bytes) + Precio (8 bytes) = 32 bytes
    public static final int TAMANO_REGISTRO = 32;
    public static final int MAX_CHARS_NOMBRE = 10;

    public static void escribirArticulo(RandomAccessFile raf, int posicion, int id, String nombre, double precio) throws IOException {
        // Saltamos directamente al byte inicial del registro deseado
        long offset = (long) posicion * TAMANO_REGISTRO;
        raf.seek(offset);

        // 1. ID: 4 bytes
        raf.writeInt(id);

        // 2. Nombre: exactamente 10 caracteres (20 bytes) usando StringBuffer
        StringBuffer sb = new StringBuffer(nombre);
        sb.setLength(MAX_CHARS_NOMBRE); // Rellena con '\0' o recorta si supera 10 caracteres
        raf.writeChars(sb.toString());

        // 3. Precio: 8 bytes
        raf.writeDouble(precio);
    }

    public static void modificarPrecioInSitu(File archivo, int posicion, double nuevoPrecio) {
        try (RandomAccessFile raf = new RandomAccessFile(archivo, "rw")) {
            // El campo precio empieza tras el ID (4 bytes) y el Nombre (20 bytes) = +24 bytes
            long offsetPrecio = ((long) posicion * TAMANO_REGISTRO) + 24;

            System.out.println("🔪 Saltando directamente al offset del precio: " + offsetPrecio);
            raf.seek(offsetPrecio);

            double precioAntiguo = raf.readDouble(); // Lee 8 bytes y avanza el puntero 8 bytes
            System.out.printf("Precio anterior: %.2f €\n", precioAntiguo);

            // ¡IMPORTANTE! readDouble() avanzó el cabezal. Debemos retroceder para sobrescribir:
            raf.seek(offsetPrecio);
            raf.writeDouble(nuevoPrecio); // Sobrescribe los 8 bytes sin tocar nada más del disco
            System.out.printf("✅ Precio actualizado in-situ a: %.2f €\n", nuevoPrecio);

        } catch (IOException e) {
            System.err.println("Error en acceso aleatorio: " + e.getMessage());
        }
    }
}

```

Trabajo Síncrono en Aula: Actividades 2.2 y 2.3

Abre tu IDE y completa las **Actividades 2.2 y 2.3 del Cuaderno de Aula** para diseñar un almacén binario de longitud fija y aplicar modificaciones **in-situ** mediante el puntero `seek()`.

3.6. Serialización y Deserialización de Objetos en Java

La **Serialización** es el mecanismo nativo de Java que permite tomar un objeto vivo en la memoria RAM y convertir todo su grafo interno, atributos y referencias en un flujo lineal de bytes binarios. La **Deserialización** realiza el proceso inverso: lee esos bytes desde disco o red y reconstruye una réplica idéntica del objeto en RAM ****sin invocar a los constructores ordinarios****.

A. Requisitos de la Entidad Serializable

- **Interfaz** `java.io.Serializable` : Interfaz marcadora sin métodos que autoriza a la JVM a inspeccionar reflectivamente la clase. Si no se implementa, se lanza `NotSerializableException` .
- **Constante** `serialVersionUID` : Identificador de 64 bits (`long`) que define la versión de la clase. Si se omite, la JVM calcula un hash automático; cualquier modificación posterior en la clase alterará el hash provocando `InvalidClassException` al intentar leer ficheros previamente guardados.
- **Modificador** `transient` : Indica al motor de serialización que el campo marcado ****no debe guardarse en disco**** (contraseñas, tokens JWT, hilos `Thread` o conexiones `Connection`). Recupera el valor por defecto (`null` o `0`) al deserializarse.

B. Ficha Técnica Exhaustiva de Instrucciones de ObjectOutputStream

INSTRUCCIÓN / MÉTODO	PARÁMETROS	RETORNO	QUÉ HACE EXACTAMENTE LA INSTRUCCIÓN A BAJO NIVEL
<code>new ObjectOutputStream(out)</code>	<code>OutputStream out</code>	Instancia	1. Conecta con el flujo de salida. 2. Escribe en el archivo la cabecera mágica de 4 bytes (0xAC 0xED 0x00 0x05). 3. Inicializa la tabla de referencias internas de objetos.
<code>oos.writeObject(Object obj)</code>	<code>Object obj</code>	<code>void</code>	1. Inspecciona la clase y sus atributos por reflexión. 2. Si es la primera vez que serializa esa clase, emite su descriptor (nombre, <code>serialVersionUID</code>). 3. Serializa recursivamente todos los campos no transitorios y los objetos anidados. 4. Registra un identificador interno de referencia para no duplicar objetos repetidos en el grafo.
<code>oos.writeUnshared(Object obj)</code>	<code>Object obj</code>	<code>void</code>	Escribe el objeto como una entidad totalmente nueva e independiente, **sin reutilizar punteros de caché** de referencias compartidas previas.
<code>oos.reset()</code>	Ninguno	<code>void</code>	Limpia la caché de objetos ya serializados. Si modificas un objeto en memoria y vuelves a llamar a <code>writeObject()</code> sin hacer <code>reset()</code> , Java ignorará los cambios y solo escribirá una referencia al estado antiguo.
<code>oos.flush()</code>	Ninguno	<code>void</code>	Fuerza el vaciado de los bytes en memoria intermedia hacia el archivo físico en disco.
<code>oos.close()</code>	Ninguno	<code>void</code>	Vacía el flujo, lo cierra y libera el descriptor en el sistema operativo.
<code>writeStreamHeader() (protected)</code>	Ninguno	<code>void</code>	Método interno que emite los 4 bytes de cabecera inicial 0xACED0005. Al sobrescribirlo en blanco en una subclase propia, podemos evitar la corrupción del archivo en modo append.

C. Ficha Técnica Exhaustiva de Instrucciones de ObjectInputStream

INSTRUCCIÓN / MÉTODO	PARÁMETROS	RETORNO	QUÉ HACE EXACTAMENTE LA INSTRUCCIÓN A BAJO NIVEL
<code>new ObjectInputStream(in)</code>	<code>InputStream in</code>	Instancia	1. Lee los primeros 4 bytes del archivo. 2. Verifica que coincidan con la firma mágica <code>0xACED0005</code>. Si no coinciden, lanza de inmediato <code>StreamCorruptedException</code>.
<code>ois.readObject()</code>	Ninguno	Object	1. Lee la descripción de la clase en el flujo. 2. Comprueba la coincidencia del <code>serialVersionUID</code>. 3. Reserva memoria para la instancia **sin llamar al constructor**. 4. Restaura el valor de cada atributo desde los bytes leídos. 5. Retorna la referencia genérica (requiere casting explícito). Lanza <code>EOFException</code> al alcanzar el final.
<code>ois.readUnshared()</code>	Ninguno	Object	Deserializa un objeto garantizando que crea una nueva referencia desacoplada.
<code>ois.close()</code>	Ninguno	void	Cierra el canal de lectura liberando el archivo físico.

D. El Problema del Append y la Solución con `MiObjectOutputStream`

Si intentas añadir objetos a un archivo existente usando `new ObjectOutputStream(new FileOutputStream(ruta, true))`, el constructor escribirá una **segunda cabecera mágica** (`0xACED0005`) en mitad del archivo. Al leer posteriormente con `ObjectInputStream`, tras leer el primer objeto el parser encontrará los bytes de cabecera esperando datos, colapsando con:

```
java.io.StreamCorruptedException: invalid type code: AC
```

Para solucionarlo de forma profesional, creamos la subclase `MiObjectOutputStream` sobrescribiendo `writeStreamHeader()` en blanco:

```
import java.io.*;

// Subclase que anula la escritura de la cabecera intermedia
class MiObjectOutputStream extends ObjectOutputStream {
    public MiObjectOutputStream(OutputStream out) throws IOException { super(out); }
    @Override
    protected void writeStreamHeader() throws IOException {
        // Al dejar este método vacío, NO se escribe la cabecera repetida 0xACED0005
    }
}

public class SolucionAppendObjetos {
    public static void anadirObjetoSeguro(File archivo, Object obj) {
        // Verificamos si el archivo ya existe Y contiene datos previos
        boolean yaTieneDatos = archivo.exists() && archivo.length() > 0;

        try {
            FileOutputStream fos = new FileOutputStream(archivo, true); // Modo append activado

            // Decisión inteligente de flujo:
            // - Si es la primera vez: ObjectOutputStream normal (escribe la cabecera requerida)
            // - Si ya tiene datos: MiObjectOutputStream (no escribe cabecera repetida)
            ObjectOutputStream oos = yaTieneDatos
                ? new MiObjectOutputStream(fos)
                : new ObjectOutputStream(fos);

            oos.writeObject(obj);
            oos.close();
            System.out.println("✅ Objeto añadido incrementalmente sin corromper el stream.");

        } catch (IOException e) {
            System.err.println("Error al persistir: " + e.getMessage());
        }
    }
}
```

Trabajo Síncrono en Aula: Actividades 2.4 y 2.5

Abre tu IDE y completa las **Actividades 2.4 y 2.5 del Cuaderno de Aula** para serializar entidades con atributos `transient` y superar el reto de la persistencia incremental con `MiObjectOutputStream`.

3.7. Persistencia e Intercambio en Formato XML

El formato **XML (eXtensible Markup Language)** es el estándar por excelencia para mensajería empresarial estructurada. En Java disponemos de 4 APIs oficiales para su tratamiento integral. A continuación se desglosan exhaustivamente cada una de ellas con sus instrucciones, parámetros y ejemplos funcionales.

A. Manera 1: DOM (Document Object Model) y Transformer

Concepto: Carga el documento XML íntegro en la memoria RAM construyendo un árbol jerárquico de nodos (`org.w3c.dom.*`). Es bidireccional y permite consultar, eliminar y modificar nodos a voluntad.

INSTRUCCIÓN / MÉTODO	CLASE ORIGEN	RETORNO	QUÉ HACE LA INSTRUCCIÓN EXACTAMENTE
<code>DocumentBuilderFactory.newInstance()</code>	Factoría	<code>DocumentBuilderFactory</code>	Obtiene la factoría proveedora de analizadores DOM configurada en el JDK.
<code>factory.newDocumentBuilder()</code>	<code>DocumentBuilderFactory</code>	<code>DocumentBuilder</code>	Instancia el constructor del parser DOM listo para procesar documentos.
<code>builder.parse(File f)</code>	<code>DocumentBuilder</code>	<code>Document</code>	Parsea el archivo XML y construye en RAM el árbol completo de nodos <code>Document</code> .
<code>builder.newDocument()</code>	<code>DocumentBuilder</code>	<code>Document</code>	Crea un árbol XML completamente nuevo y vacío en la memoria RAM listo para poblarlo.
<code>doc.getDocumentElement()</code>	<code>Document</code>	<code>Element</code>	Devuelve la etiqueta raíz principal de todo el documento XML.
<code>doc.normalize()</code>	<code>Document</code>	<code>void</code>	Limpia el árbol: unifica nodos de texto adyacentes y elimina espacios vacíos huérfanos.
<code>doc.createElement(String tag)</code>	<code>Document</code>	<code>Element</code>	Fabrica una nueva etiqueta XML en memoria vinculada al contexto del documento.
<code>doc.createTextNode(String texto)</code>	<code>Document</code>	<code>Text</code>	Crea un nodo de texto puro para colocar dentro de una etiqueta.
<code>elem.setAttribute(String k, String v)</code>	<code>Element</code>	<code>void</code>	Asigna o actualiza un atributo clave-valor dentro de la etiqueta de apertura.

INSTRUCCIÓN / MÉTODO	CLASE ORIGEN	RETORNO	QUÉ HACE LA INSTRUCCIÓN EXACTAMENTE
<code>elem.getAttribute(String k)</code>	Element	String	Lee el valor textual del atributo especificado.
<code>elem.getElementsByTagName(String tag)</code>	Element / Document	NodeList	Busca recursivamente todas las etiquetas coincidentes en el subárbol.
<code>padre.appendChild(Node hijo)</code>	Node	Node	Engancha el nuevo nodo como último hijo dentro de la jerarquía del padre.
<code>padre.removeChild(Node hijo)</code>	Node	Node	Desvincula y elimina el nodo hijo especificado del árbol en memoria.
<code>node.getNodeType()</code>	Node	short	Retorna el tipo de nodo (Node.ELEMENT_NODE == 1, Node.TEXT_NODE == 3).
<code>node.getTextContent()</code>	Node	String	Extrae todo el contenido de texto dentro del nodo y sus descendientes.
<code>node.setTextContent(String texto)</code>	Node	void	Sobreescribe el contenido textual del nodo.
<code>TransformerFactory.newInstance()</code>	Factoría	TransformerFactory	Crea la factoría del motor XSLT/Transformer.
<code>tf.setOutputProperty(OutputKeys.INDENT, "yes")</code>	Transformer	void	Configura el formato visual activando la sangría (*pretty-print*).
<code>tf.transform(Source src, Result res)</code>	Transformer	void	Vuelca físicamente el árbol DOM (new DOMSource(doc)) al archivo (new StreamResult(f)).

```
import java.io.File;
import javax.xml.parsers.*;
import javax.xml.transform.*;
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.stream.StreamResult;
import org.w3c.dom.*;

public class GestorCompletoDOM {

    public static void crearCatalogoXml(File destino) throws Exception {
        DocumentBuilder builder = DocumentBuilderFactory.newInstance().newDocumentBuilder();
        Document doc = builder.newDocument(); // Documento nuevo en blanco

        // 1. Crear nodo raíz:
        Element raiz = doc.createElement("catalogo");
        doc.appendChild(raiz);

        // 2. Crear producto con atributo id y etiquetas hijas
        Element producto = doc.createElement("producto");
        producto.setAttribute("id", "P-100");

        Element nombre = doc.createElement("nombre");
        nombre.setTextContent("Monitor Gaming 27 pulgadas");
        producto.appendChild(nombre);

        Element precio = doc.createElement("precio");
        precio.setTextContent("249.99");
        producto.appendChild(precio);

        raiz.appendChild(producto);

        // 3. Volcar árbol DOM a disco con sangrado de 4 espacios
        Transformer tf = TransformerFactory.newInstance().newTransformer();
        tf.setOutputProperty(OutputKeys.INDENT, "yes");
        tf.setOutputProperty("{http://xml.apache.org/xslt}indent-amount", "4");

        tf.transform(new DOMSource(doc), new StreamResult(destino));
        System.out.println("✅ Catálogo XML generado exitosamente con DOM.");
    }

    public static void modificarPreciosDOM(File origen) throws Exception {
        DocumentBuilder builder = DocumentBuilderFactory.newInstance().newDocumentBuilder();
        Document doc = builder.parse(origen);
        doc.getDocumentElement().normalize(); // Limpieza obligatoria

        NodeList productos = doc.getElementsByTagName("producto");
        for (int i = 0; i < productos.getLength(); i++) {
            Node n = productos.item(i);
            // FILTRADO Estricto: Descartar nodos de texto vacíos (#text)
            if (n.getNodeType() == Node.ELEMENT_NODE) {
                Element elem = (Element) n;
                Element elemPrecio = (Element) elem.getElementsByTagName("precio").item(0);
                double precioActual = Double.parseDouble(elemPrecio.getTextContent());
                double precioNuevo = precioActual * 1.10; // Incremento del 10%
                elemPrecio.setTextContent(String.format("%.2f", precioNuevo).replace(',', '.'));
            }
        }

        // Guardamos las modificaciones sobre el mismo archivo
        Transformer tf = TransformerFactory.newInstance().newTransformer();
        tf.setOutputProperty(OutputKeys.INDENT, "yes");
        tf.transform(new DOMSource(doc), new StreamResult(origen));
        System.out.println("✅ Precios actualizados en disco tras modificación en RAM.");
    }
}
```

```
}  
}
```

B. Manera 2: SAX (Simple API for XML)

Concepto: Analizador unidireccional y reactivo (*Push*). No construye ningún árbol en memoria; lee secuencialmente de arriba hacia abajo y dispara eventos que son procesados por los métodos de una clase que hereda de `DefaultHandler`. Es el estándar para archivos de cientos de megabytes o gigabytes donde DOM agotaría la memoria RAM.

MÉTODO CALLBACK (DEFAULTHANDLER)	PARÁMETROS	CUÁNDO SE DISPARA Y QUÉ LABOR REALIZAR
<code>startDocument()</code>	Ninguno	Se ejecuta exactamente una vez al comenzar a leer el archivo XML. Ideal para resetear listas y acumuladores.
<code>startElement(uri, localName, qName, attributes)</code>	<code>String uri,</code> <code>localName, qName,</code> <code>Attributes</code> <code>attributes</code>	Se dispara al encontrar una etiqueta de apertura (ej. <code><producto id="P-100"></code>). <code>qName</code> da el nombre de la etiqueta. Se leen atributos con <code>attributes.getValue("id")</code> y se limpia el acumulador de texto.
<code>characters(char[] ch, int start, int length)</code>	<code>char[] ch, int</code> <code>start, int length</code>	Se dispara cuando lee texto dentro de una etiqueta. ⚠️ *¡Atención!* El parser puede llamar a este método varias veces seguidas para un mismo texto; **es obligatorio acumularlo con <code>StringBuilder.append()</code> **.
<code>endElement(uri, localName, qName)</code>	<code>String uri,</code> <code>localName, qName</code>	Se dispara al encontrar la etiqueta de cierre (ej. <code></producto></code>). Momento en el que se extrae el texto del acumulador, se parsea a su tipo y se añade a la colección o informe.
<code>endDocument()</code>	Ninguno	Se ejecuta al finalizar completamente el documento. Ideal para imprimir estadísticas o cerrar bases de datos.

```

import java.io.File;
import javax.xml.parsers.SAXParser;
import javax.xml.parsers.SAXParserFactory;
import org.xml.sax.Attributes;
import org.xml.sax.helpers.DefaultHandler;

public class AnalizadorCatalogoSAX {
    public static void procesarConSAX(File archivoXml) throws Exception {
        SAXParserFactory factory = SAXParserFactory.newInstance();
        SAXParser saxParser = factory.newSAXParser();

        DefaultHandler handler = new DefaultHandler() {
            private StringBuilder sb = new StringBuilder();
            private String idActual;
            private double precioActual;
            private boolean dentroDePrecio = false;

            @Override
            public void startElement(String uri, String localName, String qName, Attributes atts) {
                sb.setLength(0); // Limpiamos acumulador
                if (qName.equalsIgnoreCase("producto")) {
                    idActual = atts.getValue("id");
                } else if (qName.equalsIgnoreCase("precio")) {
                    dentroDePrecio = true;
                }
            }

            @Override
            public void characters(char[] ch, int start, int length) {
                sb.append(ch, start, length); // Acumulación estricta de caracteres
            }

            @Override
            public void endElement(String uri, String localName, String qName) {
                if (qName.equalsIgnoreCase("precio") && dentroDePrecio) {
                    precioActual = Double.parseDouble(sb.toString().trim());
                    dentroDePrecio = false;
                } else if (qName.equalsIgnoreCase("producto")) {
                    System.out.printf(" ⚡ [SAX Event] Producto ID: %s | Precio: %.2f €\n", idActual, precioActual);
                }
            }
        };

        System.out.println("Iniciando análisis SAX con 0 MB de consumo de árbol:");
        saxParser.parse(archivoXml, handler);
    }
}

```

C. Manera 3: StAX (Streaming API for XML - Pull Parsing)

Concepto: A diferencia de SAX (donde el parser te empuja eventos de forma ininterrumpida), en StAX ****el programador tiene el control del bucle**** mediante un cursor iterativo (`XMLStreamReader` para lectura y `XMLStreamWriter` para escritura). Permite detener la lectura con un `break` cuando se encuentra el dato buscado, ahorrando tiempo de CPU.

INSTRUCCIÓN / MÉTODO	CLASE	RETORNO	QUÉ HACE EXACTAMENTE
<code>reader.hasNext()</code>	<code>XMLStreamReader</code>	<code>boolean</code>	Comprueba si quedan más eventos por procesar en el flujo del fichero.
<code>reader.next()</code>	<code>XMLStreamReader</code>	<code>int</code>	Avanza el cursor al siguiente token y devuelve su tipo (<code>START_ELEMENT</code> , <code>CHARACTERS</code> , <code>END_ELEMENT</code>).
<code>reader.getLocalName()</code>	<code>XMLStreamReader</code>	<code>String</code>	Retorna el nombre de la etiqueta XML sobre la que está posicionado el cursor.
<code>reader.getElementText()</code>	<code>XMLStreamReader</code>	<code>String</code>	**Consume automáticamente** el texto interior de la etiqueta hasta su cierre.
<code>reader.getAttributeValue(null, "id")</code>	<code>XMLStreamReader</code>	<code>String</code>	Lee de forma directa el valor del atributo especificado.
<code>writer.writeStartDocument("UTF-8", "1.0")</code>	<code>XMLStreamWriter</code>	<code>void</code>	Escribe la cabecera estándar de prólogo XML.
<code>writer.writeStartElement("producto")</code>	<code>XMLStreamWriter</code>	<code>void</code>	Escribe la etiqueta de apertura <code><producto></code> .
<code>writer.writeAttribute("id", "P-100")</code>	<code>XMLStreamWriter</code>	<code>void</code>	Inserta un atributo en la etiqueta que se encuentra abierta.
<code>writer.writeCharacters("Texto")</code>	<code>XMLStreamWriter</code>	<code>void</code>	Inserta texto plano dentro de la etiqueta.
<code>writer.writeEndElement()</code>	<code>XMLStreamWriter</code>	<code>void</code>	Cierra la última etiqueta abierta (ej. <code></producto></code>).

```
import java.io.FileInputStream;
import javax.xml.stream.XMLInputFactory;
import javax.xml.stream.XMLStreamConstants;
import javax.xml.stream.XMLStreamReader;

public class LectorStAXPull {
    public static void buscarProductoPorId(String rutaXml, String idBuscado) throws Exception {
        XMLInputFactory factory = XMLInputFactory.newInstance();
        try (FileInputStream fis = new FileInputStream(rutaXml)) {
            XMLStreamReader reader = factory.createXMLStreamReader(fis);

            while (reader.hasNext()) {
                int evento = reader.next();
                if (evento == XMLStreamConstants.START_ELEMENT) {
                    if ("producto".equalsIgnoreCase(reader.getLocalName())) {
                        String id = reader.getAttributeValue(null, "id");
                        if (idBuscado.equals(id)) {
                            System.out.println("🔍 Producto encontrado con StAX: " + id);
                            break; // ¡VENTAJA DE STAX!: Detenemos el parseo inmediatamente sin leer el resto
                        }
                    }
                }
            }
            reader.close();
        }
    }
}
```

D. Manera 4: JAXB (Jakarta XML Binding)

Concepto: Tecnología de enlace declarativo. Asocia clases POJO con etiquetas XML mediante anotaciones en el código Java. Elimina por completo la necesidad de navegar por árboles de nodos o controlar cursores:

- **Marshaller:** Convierte un Objeto Java en un archivo XML (Java $\rightarrow$ XML).
- **Unmarshaller:** Convierte un archivo XML en un Objeto Java en RAM (XML $\rightarrow$ Java).

ANOTACIÓN / INSTRUCCIÓN	DÓNDE SE UBICA	QUÉ HACE EXACTAMENTE
<code>@XmlRootElement(name = "catalogo")</code>	Sobre la clase	Define la clase como el elemento raíz de todo el documento XML .
<code>@XmlElement(name = "producto")</code>	Sobre atributo o getter	Asocia la propiedad con una etiqueta XML hija interior.
<code>@XmlAttribute(name = "id")</code>	Sobre atributo o getter	Guarda la propiedad como un atributo dentro de la etiqueta.
<code>@XmlTransient</code>	Sobre atributo o getter	Excluye el campo: no se exportará ni leerá del XML.
<code>@XmlElementWrapper(name = "lista")</code>	Sobre colecciones (List)	Crea una etiqueta contenedora envolvente alrededor de los elementos de la lista.
<code>JAXBContext.newInstance(Clase.class)</code>	Factoría	Construye el contexto JAXB analizando las anotaciones reflectivas de la clase raíz.
<code>marshaller.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true)</code>	Marshaller	Activa la sangría y los saltos de línea automáticos en el XML resultante.
<code>marshaller.marshal(objeto, file)</code>	Marshaller	Vuelca el objeto Java en el archivo XML físico en disco.
<code>unmarshaller.unmarshal(file)</code>	Unmarshaller	Lee el XML y devuelve la instancia Java poblada con todos sus datos.

```

import jakarta.xml.bind.JAXBContext;
import jakarta.xml.bind.Marshaller;
import jakarta.xml.bind.Unmarshaller;
import jakarta.xml.bind.annotation.*;
import java.io.File;

@XmlRootElement(name = "articulo")
@XmlType(propOrder = { "nombre", "precio" })
class Articulo {
    private String id;
    private String nombre;
    private double precio;

    // CONSTRUCTOR VACÍO OBLIGATORIO para que JAXB pueda instanciar la clase por reflexión
    public Articulo() {}

    public Articulo(String id, String nombre, double precio) {
        this.id = id; this.nombre = nombre; this.precio = precio;
    }

    @XmlAttribute(name = "id")
    public String getId() { return id; }
    public void setId(String id) { this.id = id; }

    @XmlElement(name = "denominacion")
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    @XmlElement(name = "pvp")
    public double getPrecio() { return precio; }
    public void setPrecio(double precio) { this.precio = precio; }
}

public class GestorJAXB {
    public static void main(String[] args) throws Exception {
        File destino = new File("datos/articulo_jaxb.xml");
        JAXBContext ctx = JAXBContext.newInstance(Articulo.class);

        // 1. MARSHALLING: Java -> XML
        Marshaller m = ctx.createMarshaller();
        m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true); // Sangría bonita
        Articulo art = new Articulo("ART-99", "Impresora Láser", 189.50);
        m.marshal(art, destino);
        System.out.println("✅ XML generado con Marshaller.");

        // 2. UNMARSHALLING: XML -> Java
        Unmarshaller u = ctx.createUnmarshaller();
        Articulo recuperado = (Articulo) u.unmarshal(destino);
        System.out.println("📄 Objeto recuperado: " + recuperado.getNombre() + " (" + recuperado.getPrecio() + " €)");
    }
}

```



Trabajo Síncrono en Aula: Actividades 3.1, 3.2, 3.3 y 3.4

Abre tu IDE y completa el Bloque 3 del **Cuaderno de Aula**: practica la navegación y volcado con DOM (3.1 y 3.2), la ingesta masiva sin RAM con SAX (3.3) y el mapeo POJO anotado con JAXB (3.4).

4 Ejercicios y Ejemplos Prácticos Graduados

Nivel 1 (Básico): Contador Estadístico con BufferedReader

Permite asimilar el control del Fin de Fichero (EOF) y la manipulación de cadenas de texto línea por línea.

ContadorPalabras.java

JAVA 21

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class ContadorPalabras {
    public static void analizarFichero(File archivo) {
        int totalLineas = 0;
        int totalPalabras = 0;

        try (BufferedReader br = new BufferedReader(new FileReader(archivo, StandardCharsets.UTF_8))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                totalLineas++;
                String lineaLimpia = linea.trim();
                if (!lineaLimpia.isEmpty()) {
                    String[] palabras = lineaLimpia.split("\\s+");
                    totalPalabras += palabras.length;
                }
            }
            System.out.println("Líneas leídas: " + totalLineas + " | Total palabras: " + totalPalabras);
        } catch (IOException e) {
            System.err.println("Fallo al leer: " + e.getMessage());
        }
    }
}
```

Nivel 2 (Intermedio): Transformador y Validador CSV con NIO.2

Combina `Files.lines()`, filtrado de expresiones regulares y escritura atómica en fichero de salida.

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.util.List;
import java.util.stream.Stream;

public class ProcesadorTransaccionesCSV {
    public static void filtrarVentasAltas(Path csvOrigen, Path txtDestino, double importeMinimo) {
        try (Stream<String> lineas = Files.lines(csvOrigen, StandardCharsets.UTF_8)) {

            List<String> aprobadas = lineas
                .skip(1) // Omitimos la cabecera CSV
                .filter(l -> {
                    String[] c = l.split(";");
                    return c.length >= 4 && Double.parseDouble(c[3]) >= importeMinimo;
                })
                .map(l -> {
                    String[] c = l.split(";");
                    return String.format("REF:%s | CLIENTE:%s | TOTAL:%.2f EUR", c[0], c[2], Double.parseDouble(c[3]));
                })
                .toList();

            Files.write(txtDestino, aprobadas, StandardCharsets.UTF_8,
                StandardOpenOption.CREATE, StandardOpenOption.TRUNCATE_EXISTING);

            System.out.println("Transacciones procesadas y guardadas: " + aprobadas.size());
        } catch (IOException e) {
            System.err.println("Error procesando CSV: " + e.getMessage());
        }
    }
}
```

Nivel 3 (Avanzado / Profesional): Actualizador Transaccional con RandomAccessFile

Modifica directamente el registro de un producto en disco sin recorrer el archivo ni crear copias temporales.

```
import java.io.File;
import java.io.IOException;
import java.io.RandomAccessFile;

public class ActualizadorStockRAF {
    // Registro: ID (int, 4 bytes) + Nombre (30 chars, 60 bytes) + Stock (int, 4 bytes) = 68 bytes
    public static final int TAM_REGISTRO = 68;

    public static boolean actualizarStock(File bdArticulos, int idProducto, int unidadesRestar) {
        try (RandomAccessFile raf = new RandomAccessFile(bdArticulos, "rw")) {
            // El ID determina la posición física exacta del registro: (id - 1) * 68
            long posRegistro = (long)(idProducto - 1) * TAM_REGISTRO;
            if (posRegistro + TAM_REGISTRO > raf.length()) {
                return false; // El producto no existe en el archivo
            }

            // Saltamos directamente a la posición del campo Stock (offset + 64 bytes)
            raf.seek(posRegistro + 64);
            int stockActual = raf.readInt();

            if (stockActual >= unidadesRestar) {
                raf.seek(posRegistro + 64); // Reposicionamos para sobrescribir
                raf.writeInt(stockActual - unidadesRestar);
                return true;
            }
            return false; // Stock insuficiente
        } catch (IOException e) {
            System.err.println("Error crítico en acceso directo: " + e.getMessage());
            return false;
        }
    }
}
```

5 Código Funcional Paso a Paso: Gestor Empresarial

La siguiente clase compila en un único componente profesional las mejores prácticas de la industria: resolución segura de rutas relativas con NIO.2, creación controlada de directorios padre, sanitización de codificaciones y lectura bufferizada con `try-with-resources`.

FileUtilsPro.java

JAVA 21

```
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.util.List;

public class FileUtilsPro {

    /**
     * Escribe un contenido textual de forma segura en disco, garantizando la existencia
     * de los directorios padre y aplicando codificación estandarizada UTF-8.
     */
    public static void guardarTextoSeguro(Path rutaFichero, String contenido) throws IOException {
        // 1. Asegurar que las carpetas contenedoras existan
        Path directorioPadre = rutaFichero.getParent();
        if (directorioPadre != null && !Files.exists(directorioPadre)) {
            Files.createDirectories(directorioPadre);
        }

        // 2. Escritura atómica con NIO.2 aplicando UTF-8
        Files.writeString(
            rutaFichero,
            contenido,
            StandardCharsets.UTF_8,
            StandardOpenOption.CREATE,
            StandardOpenOption.TRUNCATE_EXISTING,
            StandardOpenOption.WRITE
        );
    }

    /**
     * Copia un archivo de un directorio a otro aplicando reemplazo si ya existía.
     */
    public static void respaldarFichero(Path origen, Path destino) throws IOException {
        if (!Files.exists(origen)) {
            throw new NoSuchFileException("El archivo de origen no existe: " + origen);
        }
        Files.copy(origen, destino, StandardCopyOption.REPLACE_EXISTING, StandardCopyOption.COPY_ATTRIBUTES);
    }
}
```

6 Errores Habituales y Soluciones Técnicas

✗ Error 1: Fugas de Descriptores de Archivo del Sistema Operativo

¿Qué ocurre? Abrir flujos con `new FileReader()` o `new FileInputStream()` y cerrarlos manualmente al final del método con `fr.close()`.

Causa técnica profunda: Si se produce cualquier excepción antes de llegar a la línea del `close()`, el manejador queda retenido en la tabla de descriptores del kernel. En Windows el archivo queda bloqueado (imposibilitando que otro proceso lo borre) y en Linux colapsa el servidor al alcanzar el límite del sistema (*Too many open files*).

Solución profesional: Emplear siempre `try-with-resources`.

✗ Error 2: Corrupción de Índices en RandomAccessFile por Cadenas Dinámicas

¿Qué ocurre? Escribir cadenas mediante `writeUTF()` en un fichero de acceso directo asumiendo un tamaño de registro constante.

Causa técnica profunda: `writeUTF()` escribe 2 bytes de cabecera más los bytes reales del texto. Al cambiar el tamaño del String entre un empleado y otro, la fórmula matemática $(N-1) \times \text{tam}$ hace aterrizar el puntero en mitad de un dato numérico, corrompiendo la lectura.

Solución profesional: Asignar un número fijo de caracteres y rellenar los sobrantes con espacios en blanco.

✗ Error 3: El Desastre de la Doble Cabecera en la Serialización (modo append)

¿Qué ocurre? Usar `new ObjectOutputStream(new FileOutputStream(file, true))` para añadir objetos al final de un archivo serializado existente.

Causa técnica profunda: Cada constructor de `ObjectOutputStream` inyecta una cabecera de 4 bytes (`0xAC 0xED 0x00 0x05`). Al leer el segundo bloque, la JVM espera un objeto y se topa con otra cabecera mágica, lanzando `StreamCorruptedException`.

Solución profesional: Crear una subclase que anule el método `writeStreamHeader()` en modo append.

✗ Error 4: Colapso OutOfMemoryError al procesar XML Masivos con DOM

¿Qué ocurre? Parsear un documento XML de 300 MB mediante `DocumentBuilder.parse()`.

Causa técnica profunda: DOM instancia en la memoria RAM objetos Java para cada elemento, atributo, espacio en blanco y fragmento de texto. Un XML de 300 MB consume entre 1.2 y 1.8 GB de memoria Heap, colapsando la máquina virtual.

Solución profesional: Utilizar un parser reactivo orientado a eventos como **SAX** o **StAX**.

7 Buenas Prácticas de la Industria por Niveles

NIVEL DE CUMPLIMIENTO	REQUISITO TÉCNICO	JUSTIFICACIÓN PROFESIONAL
Obligatorio (Producción)	Especificar siempre <code>StandardCharsets.UTF_8</code> .	Evita el mojibake al migrar código entre entornos Windows y contenedores Linux.
Obligatorio (Producción)	Envolver todo flujo I/O en <code>try-with-resources</code> .	Garantiza la liberación inmediata de manejadores en el sistema operativo.
Obligatorio (Producción)	Usar <code>Path.of()</code> en vez de cadenas con barras directas.	Previene errores de resolución de rutas entre sistemas operativos distintos.
Altamente Recomendado	Definir siempre <code>private static final long serialVersionUID</code> .	Evita que modificaciones en la clase impidan deserializar datos previamente guardados.
Altamente Recomendado	Bufferizar flujos de entrada y salida con clases envoltorio.	Reduce drásticamente el número de operaciones I/O en disco físico acelerando el sistema.
Estándar de Arquitectura	No bloquear el hilo de ejecución principal durante I/O intensivo.	Mantiene la reactividad de la interfaz y la concurrencia de peticiones en servidores web.

8 Matrices Comparativas de Tecnologías

Tabla A: `java.io.File` frente a `java.nio.file.Path`

CARACTERÍSTICA	API CLÁSICA (<code>java.io.File</code>)	API MODERNA (<code>java.nio.file.Path & Files</code>)
Año de introducción	Java 1.0 (Año 1996)	Java 7 (NIO.2, Año 2011) / Java 11
Gestión de errores	Retorna <code>false</code> si falla (silencioso).	Lanza excepciones ricas (<code>NoSuchFileException</code> , etc.).
Enlaces simbólicos y metadatos POSIX	Soporte nulo o extremadamente precario.	Soporte completo multiplataforma (permisos, ACLs, atributos).
Operaciones atómicas	No admitidas por el API.	Admitidas con <code>StandardCopyOption.ATOMIC_MOVE</code> .
Integración con Programación Funcional	Nula.	Nativa mediante <code>Files.lines()</code> , <code>Files.walk()</code> .

Tabla B: Comparativa de Tecnologías de Persistencia en Ficheros

TECNOLOGÍA	TIPO DE FICHERO	VELOCIDAD DE ACCESO	LEGIBILIDAD	CASO DE USO IDÓNEO
Texto con NIO.2 / Buffer	Texto plano (UTF-8)	Media / Alta	Excelente (humana)	Logs, archivos de configuración, CSVs.
DataStream / Output	Binario secuencial	Muy Alta	Nula (binaria)	Series de datos numéricos y sensores.
RandomAccessFile	Binario indexado	Instantánea por ID (\$O(1)\$)	Nula (binaria)	Mini-motores de bases de datos, catálogos fijos.
Serialización de Objetos	Binario nativo Java	Alta	Nula (binaria)	Cachés en disco, persistencia de grafos complejos.
XML (DOM / SAX / JAXB)	Texto jerárquico	Media / Baja	Excelente (humana)	Factura electrónica, interoperabilidad entre lenguajes.

9 Resumen Clave de la Unidad

- **Streams:** Abstracción de tubería unidireccional. Se dividen en flujos de *Bytes* (`InputStream` / `OutputStream`) y flujos de *Caracteres* (`Reader` / `Writer`).
- **Buffer:** Imprescindible para el rendimiento; almacena 8 KB intermedios en RAM evitando peticiones microscópicas al disco.
- **NIO.2:** Representa el estándar moderno basado en `Path.of()` y métodos atómicos de la clase `Files` . Para ficheros gigantes, `Files.lines()` procesa de forma perezosa mediante Streams.
- **StandardCharsets.UTF_8:** Parámetro ineludible en toda lectura o escritura de texto para evitar la corrupción de tildes o caracteres multilingües (*mojibake*).
- **RandomAccessFile:** Permite leer y escribir en cualquier posición arbitraria mediante `seek()` ; requiere obligatoriamente que los registros tengan una longitud constante en bytes.
- **Serialización:** Almacena objetos en disco con `ObjectOutputStream` . Requiere implementar `Serializable` , declarar `serialVersionUID` y usar `transient` para omitir datos sensibles. En modo append exige una subclase que anule `writeStreamHeader()` .
- **XML:** DOM construye el árbol en memoria RAM (ideal para edición de ficheros pequeños/medianos); SAX procesa eventos al vuelo con consumo cero de memoria (ideal para ficheros gigantes); JAXB mapea automáticamente clases POJO a etiquetas XML.

10 Conceptos que Debes Dominar

- ✓ **Diferenciar conceptualmente** cuándo utilizar flujos de bytes frente a flujos de caracteres.
- ✓ **Justificar técnicamente** la necesidad de envolver flujos en buffers y usar `try-with-resources` .
- ✓ **Manejar con soltura** la API moderna Java NIO.2: creación de rutas con `Path.of()` , operaciones con `Files` y lectura perezosa con `Files.lines()` .
- ✓ **Construir programas robustos** que no dependan del Charset del sistema anfitrión, forzando `StandardCharsets.UTF_8` .
- ✓ **Calcular con precisión matemática** la posición en bytes de un registro en un `RandomAccessFile` y actualizarlo in-situ.
- ✓ **Implementar clases serializables** declarando explícitamente su `serialVersionUID` y aislando datos sensibles con `transient` .
- ✓ **Diagnosticar y resolver** el fallo de doble cabecera (`StreamCorruptedException`) al añadir objetos serializados en modo append.
- ✓ **Seleccionar el analizador XML adecuado** (DOM frente a SAX o JAXB) justificando la decisión en base a los requisitos de memoria y volumen de datos del sistema.

11 Glosario Técnico y Banco de Preguntas de Repaso

Glosario de Términos Fundamentales

- **Stream (Flujo):** Canal de comunicación unidireccional y secuencial que transporta bytes o caracteres entre un origen y un destino.
- **Búfer:** Área temporal de memoria RAM de alta velocidad diseñada para agrupar operaciones de lectura/escritura y reducir los accesos a disco físico.
- **Descriptor de Fichero (File Handle):** Identificador numérico asignado por el núcleo del sistema operativo a cada archivo abierto por un proceso.
- **Charset:** Mapa de correspondencia estandarizado entre secuencias binarias de bytes y caracteres legibles (alfanuméricos o gráficos).
- **Mojibake:** Texto corrupto e ilegible resultante de decodificar una secuencia de bytes con un Charset diferente al que se utilizó para escribirlo.
- **Evaluación Perezosa (Lazy Evaluation):** Estrategia de procesamiento donde los elementos de un flujo no se cargan en memoria hasta que la operación terminal los solicita.
- **Puntero de Fichero (File Pointer):** Cursor numérico que señala el índice exacto de byte en el que tendrá lugar la siguiente lectura o escritura en un `RandomAccessFile`.
- **serialVersionUID:** Identificador de versión de 64 bits que asegura que una clase Java coincide estructuralmente con los datos serializados que se intentan recuperar.
- **transient:** Modificador reservado en Java que indica al motor de serialización que ignore dicho atributo al guardar el objeto.
- **DOM (Document Object Model):** Especificación de parseo XML que crea en memoria RAM un árbol de objetos navegable bidireccionalmente.
- **SAX (Simple API for XML):** Parser unidireccional basado en eventos que lee XML continuo con un consumo mínimo y constante de memoria RAM.
- **Marshalling / Unmarshalling:** Proceso de transformar un grafo de objetos Java en un documento XML (Marshalling) y el proceso inverso (Unmarshalling) mediante JAXB.

Banco de 15 Preguntas de Repaso (Sin Respuesta Inmediata)

Utiliza estas 15 preguntas abiertas para comprobar tu nivel de comprensión técnica antes de enfrentarte a las pruebas de evaluación y exámenes del módulo:

Pregunta 1: Explica detalladamente la diferencia arquitectónica entre la jerarquía de clases que descende de `InputStream` / `OutputStream` y la que descende de `Reader` / `Writer`. ¿En qué situación técnica cometerías un error grave si utilizas un `Reader` para copiar una imagen JPEG?

Pregunta 2: ¿Qué ventaja aporta envolver un `FileReader` en un `BufferedReader`? Explica qué ocurre a nivel de llamadas al sistema operativo y qué método de alto nivel añade para facilitar la lectura.

Pregunta 3: Un programador junior afirma: "Siempre utilizo `Files.readAllLines()` porque es más cómodo y escribe menos código". ¿Bajo qué circunstancias de producción esta afirmación provocará la caída total del servidor de aplicaciones? ¿Qué método alternativo de NIO.2 debes utilizar en su lugar y por qué?

Pregunta 4: ¿Por qué es estrictamente obligatorio cerrar el flujo generado por `Files.lines()` mediante un bloque `try-with-resources`, mientras que un Stream generado a partir de un `ArrayList` no lo requiere?

Pregunta 5: Explica qué es el *mojibake*. Pon un ejemplo de qué ocurre con la letra 'ñ' si un archivo guardado con codificación Windows-1252 es abierto por un programa Java que utiliza UTF-8 por defecto. ¿Cómo se soluciona en el código?

Pregunta 6: Menciona tres diferencias fundamentales entre la antigua clase `java.io.File` y la interfaz moderna `java.nio.file.Path` combinada con `java.nio.file.Files`.

Pregunta 7: En un fichero de acceso aleatorio gestionado con `RandomAccessFile`, si cada registro ocupa exactamente 68 bytes, escribe la instrucción exacta en Java para posicionar el puntero en el inicio del registro número 12.

Pregunta 8: ¿Por qué es indispensable que los registros de un `RandomAccessFile` tengan una longitud fija en bytes? ¿Qué problema ocurriría si guardas cadenas de texto usando `writeUTF()` sin normalizar su tamaño?

Pregunta 9: Si modificas los 8 bytes de un salario en la mitad de un archivo mediante `RandomAccessFile.seek()` y `writeDouble()`, ¿se pierden o desplazan los datos que vienen después en el fichero? Justifica cómo gestiona el disco esta operación.

Pregunta 10: ¿Para qué sirve el identificador `serialVersionUID` en una clase que implementa `Serializable`? ¿Qué ocurre si no lo declaras de forma explícita y posteriormente agregas un método a la clase?

Pregunta 11: ¿Qué utilidad tiene la palabra clave `transient`? Pon dos ejemplos concretos de atributos de una clase en un sistema empresarial que deban ser declarados obligatoriamente como `transient`.

Pregunta 12: Explica la causa física a nivel de bytes del error `java.io.StreamCorruptedException: invalid type code: AC` al añadir objetos con `ObjectOutputStream` en modo append. ¿Cómo se soluciona técnicamente?

Pregunta 13: Te encargan desarrollar un microservicio para procesar catálogos XML de proveedores que pesan más de 800 Megabytes cada uno. ¿Qué tecnología de procesamiento XML elegirías (DOM o SAX) y por qué motivo técnico de uso de memoria?

Pregunta 14: En el procesamiento con SAX, ¿cuándo se ejecutan los métodos `startElement()`, `characters()` y `endElement()` del `DefaultHandler`?

Pregunta 15: ¿Qué diferencias existen entre los procesos de *Marshalling* y *Unmarshalling* en la biblioteca JAXB y qué anotación es indispensable en la clase raíz?